Systems Programming and Computer Architecture

Laurin Seeholzer

October 28, 2026

Contents

1 C Programming **5**
- 1.1 History and Toolchain . . . 5
- 1.2 Control flow . . . 5
- 1.3 Basic types in C . . . 6
- 1.4 Operators . . . 7
- 1.5 Arrays . . . 7
- 1.6 Preprocessing . . . 8
- 1.7 Modularity . . . 9
- 1.8 Assertions . . . 10
- 1.9 Style . . . 10

2 Integers **11**
- 2.1 Operations . . . 11

3 Pointers **13**
- 3.1 Memory Layout and Virtual Address Space . . . 13
- 3.2 Pointer Fundamentals . . . 13
- 3.3 Pointer Operations . . . 14
- 3.4 Function Pointers . . . 14

4 Dynamic Memory Allocation **15**
- 4.1 Data Structures . . . 16
- 4.2 Memory Allocators . . . 17
  - 4.2.1 Tracking Free Blocks . . . 18
- 4.3 Memory Pitfalls . . . 20

5 Design and Debugging **22**
- 5.1 Defects, Errors, and Failures . . . 22
- 5.2 Testing Principles . . . 22
- 5.3 Debugging Advice and Strategies . . . 23

6 Basic x86 Architecture and Assembly **24**
- 6.1 ISA and Machine State . . . 24
- 6.2 Registers and Data Movement . . . 25
- 6.3 Arithmetic and Logical Operations . . . 26
- 6.4 Condition Codes, `setX`, and `jX` . . . 26

7 Compiling C Control Flow and Procedures **28**
- 7.1 Conditionals: Branching vs. Conditional Moves . . . 28
- 7.2 Loops and Switch Statements . . . 28
- 7.3 Procedures and the Calling Convention . . . 29
- 7.4 Stack Frame Optimizations and Variadic Functions . . . 30

8 Compiling C Data Structures **32**
- 8.1 Basic Data Types and Machine Representation . . . 32
- 8.2 One-Dimensional Arrays . . . 32
- 8.3 Nested Arrays . . . 33
- 8.4 Multi-Level Arrays . . . 33

8.5 Structures and Data Alignment 33
8.6 Unions 34

1 1§

1 C Programming

1.1 History and Toolchain

C is very fast, offers preprocessing and is close to metal. What you won't find in C are objects, classes, traits, features, methods, interfaces, fancy types AND exception handling.

Important 1.1. Most importantly C does not offer memory managment, it uses pointers that offer direct access to memory adresses.

We'll use C99, the most commonly used variant, in this course.

Code Example 1.2

```
#include <studio.h>

int main(int argc, char *argv[])
{
    printf("hello, world\n");
    return 0;
}
```

1.2 Control flow

Code Example 1.3

```
if (Boolean expression) Statement_when_true
    else Statement_when_false

switch (Integer expression) {
    case Constant_1: Statement; break;
    ...
    default: Statement; break;
}

for (Initial; Test; Increment) Statement

while (Boolean expression) Statement

do Statement while (Boolean expression)

goto Label

printf()

return (Expression)
```

Important 1.4. Don't use goto: It's almost never a good idea, usually argued on

performance. (Exception: goto can be used to break out of multiple nested loops or to cleanup nested code.)

1.3 Basic types in C

Code Example 1.5

```
{static, const} type vairable = value;
```

Remark 1.6. The prefix **const** before a variable ensures it can not be changed lateron.

Definition 1.7. Scope: The scope of a variable in C is the Block it is definied in (lexically typed).

Remark 1.8. The prefix **static** before a variable definition ensures the value persists between function calls. Note that static limits the scope of a variable to the file/compilation unit.

Definition 1.9. Integers & Floats: Size of these differ from system to system

- **char** is always 1 Byte
- **int & float** are always 4 Bytes
- **long long & double** are always 8 Bytes

Definition 1.10. Booleans: Boolean values in C are integers where zero = false, non-zero = true

Definition 1.11. Void: Void is a type without a value and can be used for raw memory pointers or functions without returnvalue.

Definition 1.12. enums: used to enumerate/sequentially assign values to variables.

Code Example 1.13

```
eum { ThisIsZero, ThisIsOne, ThisIsTwo} numbers;
```

1.4 Operators

Code Example 1.14

```
() []
! ~ ++ -- + - * & (type) sizeof
* / %
+ -
<< >>
< <= > >=
== !=
&
^
|
&&
||
?:
= += -= *= /= %= &= ^= |= <<= >>=
,
```

Remark 1.15. Early termination in C means expressions such as A & B never evaluate B if A is already evaluated to false.

Definition 1.16. Ternery Operator: ternery operators allow for concise conditional assignment.

Code Example 1.17

```
int a = {test}? ifTrue : ifFalse;
```

Definition 1.18. Assignment Operator: In C an assignment to a variable is not just a statement its an expressison that evalueates to the value that is assigned.

Definition 1.19. Post- & Pre-Increment/Decement: As in many languages the difference between $a++$ and $++a$ is the value of the expression (before/after incrementing)

Important 1.20. Casting of most types to most other types in possible in C. What exactly happens is very complicated and determined by the types involved.

1.5 Arrays

Definition 1.21. Arrays are finite vectors of variables which all have the same type.

Important 1.22. The C-compiler does not check array bounds, always check them manualy!

Remark 1.23. Arrays do not have to be initialized when they are definied.

Code Example 1.24

```
int x[N] //uninitialized
int x[N] {} //initialized with zeros
```

Remark 1.25. Strings are initialized as an array of chars

Code Example 1.26

```
char s1[6] = "hello";
char s2[6] = {'h', 'e' , 'l', 'l', 'o', 0};
```

Remark 1.27. There are many libraries, to copy, compare, concatenate... strings in C. The language itself does not support any of these operations.

1.6 Preprocessing

Definition 1.28. Preprocessing replaces tokens in a C source file before compilation occurs. Text replacements and macros are managed using directives beginning with #.

Code Example 1.29

```
#define FOO BAZ       // Replaces FOO with BAZ
#define BAR(x) (x+3)  // Replaces BAR(x) with (x+3)
#undef FOO            // Undefines FOO; no replacements after this
    point
#define QUX           // Defines QUX as a null string
```

Remark 1.30. Complex multi-statement macros wrap code in a `do { ... } while(0)` loop to safely absorb trailing semicolons and prevent syntax errors when used in control flow statements (such as `if-else`).

Remark 1.31. Conditional compilation directives include or exclude blocks of code based on conditions evaluated by the preprocessor:

Code Example 1.32

```
#if expression
    ...
#elif expression
    ...
#else
    ...
#endif

#ifdef FOO     // Shorthand for #if defined(FOO)
#ifndef BAR    // Shorthand for #if !defined(BAR)
```

Remark 1.33. Macro parameters can be manipulated using special operators:

- `#c` (Stringifying): Converts a macro parameter into a string literal.
- `x ## y` (Token Pasting / Concatenation): Merges two tokens into a single token at compile-time.

Remark 1.34. The C preprocessor provides several standard predefined macros:

- `__FILE__`: Name of the current source file.
- `__LINE__`: Current line number.
- `__DATE__` / `__TIME__`: Date and time when the preprocessor runs.
- `__STDC__` / `__STDC_VERSION__`: Standard C compliance and version info.

1.7 Modularity

Definition 1.35. Declaration vs. Definition:

- A **declaration** specifies that an entity (function prototype, type, or global variable) exists somewhere without allocating memory for it.
- A **definition** provides the actual implementation or reserves memory for the variable/function.

Definition 1.36. A **module** consists of a public interface defined in a header file (`module.h`) containing declarations and macros, and an implementation in a source file (`module.c`) containing definitions and private declarations.

Remark 1.37. Declarations can be annotated with linkage modifiers:

- `extern`: The definition is located elsewhere (in another object module or compilation unit).
- `static`: Limits the visibility/scope of a function or global variable strictly to the current compilation unit.

Code Example 1.38

```
/* file.h */
#ifndef FILE_H
#define FILE_H

// Declarations, function prototypes, typedefs, and macros

#endif // FILE_H
```

1.8 Assertions

Definition 1.39. An **assertion** (`assert(expression)`) is a macro that evaluates a scalar condition at runtime. If the condition evaluates to false (`0`), it prints diagnostic details (file name, line number, expression) to standard error and terminates execution via `abort()`.

Remark 1.40. Assertions are tools for developers to detect program bugs and verify invariants during development. They should not be used to catch standard runtime or user errors (such as failed file opens). Assertions can be completely disabled in production builds by defining the `NDEBUG` macro (`-DNDEBUG`).

1.9 Style

Remark 1.41. Key guidelines for writing clean C code:

- Maintain clear formatting and consistent indentation to highlight block structure.
- Use explicit parentheses when in doubt to avoid operator precedence ambiguities.
- Use descriptive names for global entities and short names for local variables.
- Eliminate magic numbers using `const`, `enum`, or `#define` constants.
- Avoid function-like macros where regular inline functions suffice.

2 Integers

Lemma 2.1. Integers in C can be represented as **signed** (using two's complement) or **unsigned** (as a standard binary string).

Definition 2.2. Two's Complement: The most significant bit (MSB) has a negative weight (-2^{w-1}), while all other bits have positive weights. This allows representation of both positive and negative integers. To negate a number, invert all bits and add 1 ($\sim x + 1$).

Remark 2.3. Unsigned integers range from 0 to $2^w - 1$ while Signed integers range from -2^{w-1} to $2^{w-1} - 1$

Lemma 2.4. Casting between signed and unsigned does not change the underlying bit representation, only how the bits are interpreted.

Remark 2.5. In operations involving both a signed and an unsigned integer, the signed integer is automatically implicitly cast to unsigned.

2.1 Operations

Remark 2.6. Logic operations (`&`, `|`, `^`, `~`) operate bitwise. They behave as expected and return a signed integer if both operands are signed.

Definition 2.7. Shift Operations:

- **Left Shift** (`x << y`): Shifts bits left by y positions, filling with 0s on the right. Equivalent to multiplying by 2^y.
- **Logical Right Shift** (`x >> y`): Used for unsigned integers. Shifts right by y positions, filling with 0s on the left.
- **Arithmetic Right Shift** (`x >> y`): Used for signed integers. Shifts right by y positions, filling with the sign bit on the left.

Lemma 2.8. Addition: Signed and unsigned addition are performed at the bit level identically. If the result exceeds w bits, it overflows, and the extra bits are truncated.

Lemma 2.9. Multiplication: Exact multiplication requires $2w$ bits. In C, results are truncated to w bits. The bit-level operations for signed and unsigned multiplication yield the same truncated bit pattern.

Remark 2.10. Using Shift for Multiplication/Division:

- Multiplication by a power of 2 ($x \cdot 2^y$) can be optimized as `x << y`.
- Division by a power of 2 ($x/2^y$) can be optimized as `x >> y`. For signed negative integers, a bias is added before shifting to round towards zero: `(x + (1 << y) - 1) >> y`.

Lemma 2.11. Division: Integer division truncates towards zero. It is an expensive operation compared to addition or multiplication.

3 Pointers

3.1 Memory Layout and Virtual Address Space

Definition 3.1. The operating system provides each process with its own private virtual address space. Memory is organized into specific segments:

- **Kernel Virtual Memory:** Memory at the top of the address space that is inaccessible to user code.
- **User Stack:** Dynamically created region at runtime for managing function calls, growing downward.
- **Shared Libraries & Heap:** Memory-mapped region for shared libraries and dynamically allocated memory created via `malloc`.
- **Read/Write Segment:** Stores initialized and uninitialized global and static variables (`.data`, `.bss`) loaded from the executable.
- **Read-Only Segment:** Contains executable code and read-only constants (`.init`, `.text`, `.rodata`) loaded from the executable.

Remark 3.2. Set-up code allocates a stack frame upon entering a procedure to store local variables, return information, and temporary space. The space is deallocated by finish code upon return.

3.2 Pointer Fundamentals

Definition 3.3. A pointer is a variable whose value is the virtual memory address of another variable.

- **Address-of Operator (`&`):** Returns the memory address of a variable.
- **Dereference Operator (`*`):** Accesses or modifies the memory location referred to by a pointer.
- **Double Pointer:** A variable holding the memory address of another pointer (e.g., `int **dp`).
- **NULL Pointer:** A pointer assigned to address `0` (or `0x00000000`), representing a guaranteed invalid memory location.

Important 3.4. Any attempt to dereference a `NULL` pointer results in a segmentation fault.

Code Example 3.5

```
1 int x = 42;
2 int *p = &x;   // p holds the address of x
```

```
int **dp = &p; // dp holds the address of p
*p = 99;       // dereferences p to assign 99 to x
```

3.3 Pointer Operations

Definition 3.6. Pointer Arithmetic: Adding or subtracting an integer n to a pointer shifts the target memory address by $n \times$ sizeof(type) bytes.

Definition 3.7. Casting: Type casting a pointer reinterprets the byte-level data at that memory address according to the new type without changing the address itself.

Code Example 3.8

```
int arr[3] = {1, 2, 3};
int *int_ptr = &arr[0];
int_ptr += 1; // moves forward by sizeof(int) bytes to &arr[1]

char *char_ptr = (char *) int_ptr; // casts int* to char*
char_ptr += 1; // moves forward by sizeof(char) byte
```

3.4 Function Pointers

Definition 3.9. A function pointer stores the executable code address of a function, allowing functions to be passed as arguments or stored inside structures to implement polymorphism and object-oriented patterns.

Code Example 3.10

```
// Pointer to a function taking (int*, char) and returning int
int (*func)(int *, char);

// Example struct containing function pointers (vtable style)
struct file_operations {
ssize_t (*read) (struct file *, char *, size_t, loff_t *);
ssize_t (*write) (struct file *, const char *, size_t, loff_t *);
};
```

4 Dynamic Memory Allocation

Definition 4.1. Static vs Automatic Memory Allocation: Global variables are statically allocated when the program is loaded and deallocated when it exits, while function variables are automatically allocated on function call and deallocated on return.

Remark 4.2. Note that because automatically allocated memory is created on function calls, it is stored on the stack.

Remark 4.3. Often we need a larger amount of memory space that persists across multiple function calls without lasting the entire program lifetime.

Definition 4.4. Dynamic Memory Allocation: We can use allocation functions (such as `malloc`) to explicitly request dynamic memory blocks on the heap.

Code Example 4.5

```
1 char *a = (char *) malloc(10 * sizeof(char)); // Allocates
     uninitialized memory for 10 characters
2 char *zeros = (char *) calloc(10, sizeof(char)); // Allocates zero-
     initialized memory for 10 characters
3 free(a); // Frees allocated memory block
4
5 zeros = (char *) realloc(zeros, 20 * sizeof(char)); // Resizes
     allocated memory block to 20 characters
```

Definition 4.6. Memory Corruption: Occurs when a program unintentionally reads, modifies, or overwrites memory contents outside the bounds of an allocated block, or writes to memory that has already been deallocated or belongs to allocator metadata. This causes unpredictable program behavior, silent data corruption, or segmentation faults.

Definition 4.7. Memory Leaks: Occur when dynamically allocated heap memory is no longer referenced by any active pointer in the application but has not been returned to the heap via `free()`. Over time, uncollected memory accumulates and exhausts available memory.

Important 4.8. In C, memory management is strictly manual: every byte allocated on the heap must eventually be deallocated exactly once. Failing to properly manage memory boundaries and lifetimes leads to either memory corruption or memory leaks.

4.1 Data Structures

Definition 4.9. Structured Data is a custom type containing multiple data fields of different types. It is defined using the `struct` keyword.

Code Example 4.10

```
struct myData {
    int dataField;
    char dataName;
};
```

Definition 4.11. Unions overlay multiple member fields in a single shared memory location. Only one member value should be stored/interpreted at a time, and C performs no runtime tracking of the active member type.

Code Example 4.12

```
union currentTime {
    char stringName[10];
    int intTime;
    long longTime;
};
```

Remark 4.13. We can use `typedef` to define type aliases. Mixing struct tags and typedef aliases with the same name can cause confusion.

Remark 4.14. Tags (e.g., `struct tag {...}`) belong to a separate namespace from typenames; consequently, a typename and a tag can share the same identifier.

Remark 4.15. Structs and pointers allow building **dynamic data structures**. Using `void` pointers allows elements to be generic.

Code Example 4.16

```
struct listNode {
    void *value;
    struct listNode *next;
```

```
};

struct listNode *myList = NULL;
```

4.2 Memory Allocators

Definition 4.17. Implicit vs. Explicit Allocators:

- **Explicit Allocator:** The application program explicitly allocates and deallocates memory blocks (e.g., `malloc` and `free` in C, `new` and `delete` in C++).
- **Implicit Allocator (Garbage Collector):** The application explicitly requests memory allocation, but deallocation is handled automatically by the runtime system when a block is no longer in use or reachable (e.g., in Java, Python, Go, and Lisp).

Remark 4.18. Allocator Constraints and Requirements:

- **Arbitrary request sequences:** The allocator cannot make assumptions about the ordering or interleaving of allocation and deallocation requests.
- **Immediate response:** Requests must be serviced immediately; the allocator cannot buffer or reorder requests to optimize placement.
- **Heap storage only:** Any metadata structures used by the allocator must reside directly within the heap itself.
- **Alignment requirements:** Blocks must satisfy architecture-specific alignment boundaries (e.g., 8-byte alignment on 32-bit systems, 16-byte alignment on 64-bit systems).
- **Non-relocatable blocks:** Once allocated, blocks cannot be moved, compacted, or relocated by the allocator.

Definition 4.19. Performance Goals: Allocator efficiency is governed by two competing metrics: Throughput and memory utilization.

Definition 4.20. Internal vs. External Fragmentation:

- **Internal Fragmentation:** Occurs when an allocated block is larger than the requested payload. Caused by allocator metadata overhead (headers, footers), alignment padding, or minimum block size constraints. It depends solely on individual request sizes and allocator overhead, making it easy to quantify.
- **External Fragmentation:** Occurs when there is sufficient aggregate free memory in the heap to satisfy an allocation request, but no single contiguous free block is large enough. It depends on the future history of requests and

is significantly harder to measure and avoid.

Definition 4.21. Block Structure and Header: Because `free(ptr)` receives only a pointer without the block size, each block stores metadata preceding the payload:

- **Header:** A single word (4 or 8 bytes) placed immediately before the user payload containing:
  - **Block Size:** The total size of the block (including header, payload, and padding).
  - **Allocated Bit:** Since blocks are aligned to multiples of 8 (or 16) bytes, the lowest 3 (or 4) bits of any valid block size are always zero. The allocator repurposes the least significant bit (LSB) as an allocation flag: `1` for allocated, `0` for free.
- **Payload:** The user-accessible memory returned by `malloc`.
- **Padding:** Optional unused space to satisfy alignment constraints.

4.2.1 Tracking Free Blocks

Definition 4.22. Implicit Free List: All blocks (both allocated and free) are chained together implicitly through the size fields in their headers. Traversing blocks requires scanning sequentially from the beginning of the heap.

Definition 4.23. Explicit Free List: Links only the *free* blocks into an explicit doubly linked list by storing `prev` and `next` pointers inside the unused payload area of each free block.

Definition 4.24. Segregated Free Lists: Maintains an array of distinct free lists partitioned by size class (e.g., small exact classes like $16, 24, 32$ bytes, followed by power-of-two ranges like 33–64, 65–128, etc.).

Definition 4.25. Coalescing: When a block is freed, it must be merged with adjacent free blocks to prevent false fragmentation (consecutive small free blocks failing to satisfy a larger request).

Definition 4.26. Garbage Collection (GC): An automated runtime technique that dynamically reclaims heap memory that will no longer be accessed by the application.

Remark 4.27. Memory as a Directed Graph: Garbage collection models heap memory as a directed reachability graph.

Definition 4.28. Classical Garbage Collection Algorithms:

- **Reference Counting:** Each block maintains a counter tracking incoming references. The block is freed immediately when the counter drops to zero. Cannot handle cyclic references without special cycle detection.
- **Mark-and-Sweep:** Tracing collector that identifies all reachable nodes by traversing from roots (mark phase), and then sweeps through all heap blocks to deallocate unmarked nodes (sweep phase).
- **Stop-and-Copy (Copying GC):** Splits heap memory into two regions (from-space and to-space). Reachable objects are copied contiguously into to-space, automatically compacting memory and eliminating external fragmentation.
- **Generational Collection:** Groups objects by lifetime into generations (young and old), collecting the short-lived young generation frequently and promoting surviving objects to the older generation.

Example 4.29. Mark-and-Sweep in Action:

- **Mark Phase:** Begins at the root set and traverses all pointers (e.g., via depth-first search), setting a mark bit in the header of each reached block.
- **Sweep Phase:** Performs a linear scan over all heap blocks. If a block's mark bit is clear, it is freed; if set, its mark bit is reset to 0 in preparation for the next cycle.

Code Example 4.30

```
// Pseudocode for Mark-and-Sweep Collection
void mark(ptr p) {
    if ((b = block_of(p)) != NULL && !is_marked(b)) {
        set_marked(b);
        for (each pointer child in payload(b))
            mark(child);
    }
}

void sweep() {
    ptr p = heap_start();
    while (p < heap_end()) {
        if (!is_marked(p)) {
            if (is_allocated(p))
                free(p);
        } else {
            clear_marked(p);
        }
        p = next_block(p);
    }
}
```

Remark 4.31. Conservative GC for C: Because C is not type-safe, the runtime cannot distinguish whether an arbitrary memory word is a pointer or an integer. Conservative collectors (such as the Boehm-Demers-Weiser collector) assume that any word in the stack, registers, or heap whose numerical value corresponds to a valid heap address is a pointer, marking the referenced block as reachable to prevent premature freeing.

4.3 Memory Pitfalls

Remark 4.32. Because C allows direct pointer manipulation without automatic bounds checking or garbage collection, memory errors are common, difficult to diagnose, and can lead to severe security vulnerabilities.

Definition 4.33. Common Memory Pitfalls:

- **Dereferencing Bad Pointers:** Dereferencing an uninitialized pointer, a `NULL` pointer, or an arbitrary invalid address. This causes a segmentation fault (`SIGSEGV`) or corrupts arbitrary memory.
- **Reading Uninitialized Memory:** Assuming memory returned by `malloc()` is zero-filled. `malloc()` leaves the allocated block uninitialized with indeterminate garbage values. Memory must be initialized explicitly or allocated with `calloc()`.
- **Overwriting Memory (Buffer Overflow):** Writing past the boundaries of an allocated block due to off-by-one errors (e.g., `<=` instead of `<`), missing string null terminators, or calculating incorrect allocation sizes. This corrupts adjacent variables or heap metadata.
- **Referencing Nonexistent Variables:** Returning or storing the address of a local automatic stack variable. When the function returns, its stack frame is popped; the pointer becomes dangling, and subsequent function calls overwrite the data.
- **Freeing Blocks Multiple Times (Double Free):** Calling `free(p)` twice on the same pointer without an intervening allocation. This corrupts the allocator's free list data structures and can create exploitable security vulnerabilities.
- **Use-After-Free:** Accessing or modifying memory after it has been freed. The allocator may reallocate that block for another purpose, leading to silent data corruption or crashes.
- **Failing to Free Blocks (Memory Leaks):** Allocating memory on the heap and discarding all pointers to it without calling `free()`. Over time, leaked memory accumulates, exhausting system memory.

Remark 4.34. Best Practices for Safe Memory Management:

- Always set freed pointers to `NULL` (`free(p); p = NULL;`), as calling `free(NULL)` is guaranteed to be safe and harmless in C.
- Use automated memory debugging and dynamic analysis tools such as **Valgrind** or AddressSanitizer (`-fsanitize=address`) during development to detect leaks and invalid accesses at runtime.

5 Design and Debugging

5.1 Defects, Errors, and Failures

Definition 5.1. Defect, Error, and Failure Taxonomy:

- **Defect (Fault / Bug):** An imperfection, flaw, or mistake in the source code or design created by the programmer (e.g., an off-by-one loop condition, an uninitialized variable, or a missing boundary check).
- **Error:** An erroneous or invalid internal runtime state of the program resulting from the execution of a defect (e.g., an internal variable holding an illegal value or corrupted memory on the heap). An error is internal and may not yet be observable from outside the system.
- **Failure:** An externally observable divergence of the system's behavior from its specification or expected behavior (e.g., a segmentation fault / crash, incorrect output displayed to the user, an infinite loop, or data corruption).

Remark 5.2. The Fault Propagation Chain:

$$\text{Defect (Code)} \xrightarrow{\text{Execution}} \text{Error (State)} \xrightarrow{\text{Propagation}} \text{Failure (Symptom)}$$

5.2 Testing Principles

Definition 5.3. Core Testing Strategies:

- **Test Early:** Testing should begin alongside development rather than being deferred until the end. Finding and resolving defects early reduces debugging complexity and cost.
- **Test Incrementally:** Build and test software in small, verifiable steps. When code is tested after every small addition, newly introduced bugs are localized to the most recent changes.
- **Test Small, Simple Units:** Isolate individual functions and components (unit testing) to verify their behavior independently before combining them into complex systems.
- **Test Boundary Conditions (Corner Cases):** Software bugs disproportionately cluster around edges and extreme limits. Thorough test suites must evaluate:

Remark 5.4. Dijkstra's Aphorism on Testing:

> *"Program testing can be used to show the presence of bugs, but never to show their absence!"* — Edsger W. Dijkstra (1970)

No matter how many test cases pass, testing cannot formally prove that a program is defect-free because testing all possible inputs and system states is computationally infeasible. Testing must be combined with disciplined design, modularity, assertions, and invariant verification.

5.3 Debugging Advice and Strategies

Definition 5.5. Systematic Debugging Guidelines:

- **Examine the Most Recent Change:** If a previously working system breaks, the bug almost certainly lies within the most recent modifications. Use version control tools (`git diff`, `git log -p`, `git bisect`) to isolate changes.
- **Simplify the Input (Minimal Reproducible Example):** Reduce large, complex failing inputs to the smallest, simplest possible test case that still triggers the failure. Eliminating irrelevant operations simplifies tracing program state.
- **Inspect the Stack Trace:** Upon a crash or assertion failure, use a debugger (e.g., `gdb` command `backtrace` or `bt`) or core dump to identify the exact call chain, active stack frames, and parameter values at the point of failure.
- **Sanity Check Intermediate Results:** Formulate concrete hypotheses about program state and verify them at key execution milestones using assertions (`assert()`), logging, or debugger watchpoints. Verify preconditions, postconditions, and loop invariants.
- **Understand the Root Cause:** Avoid superficial "band-aid" fixes (such as blindly changing `<` to `<=`, adding arbitrary `+1` offsets, or silencing warnings). Only modify code once the exact failure mechanism and the rationale for the fix are fully understood.

Remark 5.6. Essential Systems Debugging Tools:

- **GDB (GNU Debugger):** Interactive debugger to step through execution (`step`, `next`), inspect variable values (`print`), set breakpoints/watchpoints, and inspect crash backtraces (`bt`).
- **Valgrind (Memcheck):** Dynamic memory analysis tool detecting memory leaks, invalid heap reads/writes, and use-after-free bugs.
- **Sanitizers (GCC / Clang):** Compile-time instrumentation adding automated checks:
  - AddressSanitizer (`-fsanitize=address`): Catches buffer overflows and use-after-free bugs.
  - UndefinedBehaviorSanitizer (`-fsanitize=undefined`): Catches undefined behavior (e.g., signed overflow, invalid shifts, null dereferences).

6 Basic x86 Architecture and Assembly

6.1 ISA and Machine State

Definition 6.1. ISA vs. Microarchitecture: The **Instruction Set Architecture (ISA)** defines the programmer-visible contract between software and hardware (instructions, registers, memory model), independent of the underlying **microarchitecture** (hardware implementation like caches and pipelines).

Definition 6.2. CISC vs. RISC:

- **CISC (Complex Instruction Set Computer, e.g., x86):** Variable-length instructions (1–15 bytes), arithmetic instructions can access memory directly, and explicit hardware stack operations (`push`, `pop`).
- **RISC (Reduced Instruction Set Computer, e.g., ARM, RISC-V):** Fixed-size 32-bit instructions, load-store architecture (only load/store instructions access memory), and larger register files.

Remark 6.3. Evolution of x86: Evolved with backward compatibility from 16-bit (**8086**) to 32-bit (**IA32**) and 64-bit (**x86-64**). Due to the 16-bit origin, Intel terminology defines a "word" as 16 bits (2 bytes).

Definition 6.4. Programmer-Visible State (x86-64):

- **Program Counter (`%rip`):** 64-bit address of the next instruction to execute.
- **Register File:** 16 general-purpose 64-bit integer registers and vector registers (`%xmm`, `%ymm`) for floating-point/SIMD.
- **Condition Codes:** Single-bit status flags (`CF`, `ZF`, `SF`, `OF`) updated by arithmetic operations.
- **Memory:** Flat byte-addressable virtual address space storing code, data, and the stack.

Remark 6.5. Machine Data Types & Operations: Assembly has no aggregate types (arrays, structs). It operates only on primitive integer data (1, 2, 4, or 8 bytes), floating-point values, and untyped memory addresses. Instructions fall into three categories:

1. **Data movement** between registers and memory (`mov`, `push`, `pop`).
2. **Arithmetic and logical operations** (`add`, `sub`, `imul`, shifts, bitwise ops).

3. **Control transfer** (`jmp`, conditional branches `jX`, `call`, `ret`).

Remark 6.6. AT&T Syntax (GCC/Linux default): Operands flow from source to destination:

`instr`⟨size⟩ Source, Destination

Registers use `%` (`%rax`), and immediates use `$` (`$42`). Size suffixes: **`b`** (1B, `char`), **`w`** (2B, `short`), **`l`** (4B, `int`), **`q`** (8B, `long`/pointer).

6.2 Registers and Data Movement

Definition 6.7. x86-64 General-Purpose Registers: Lower sub-registers (32-bit, 16-bit, 8-bit) can be accessed directly:

64-bit (`q`)	32-bit (`l`)	16-bit (`w`)	8-bit (`b`)	Convention / Role
`%rax`	`%eax`	`%ax`	`%al`	Return value
`%rbx`	`%ebx`	`%bx`	`%bl`	Callee-saved
`%rcx`	`%ecx`	`%cx`	`%cl`	4th argument, shift count
`%rdx`	`%edx`	`%dx`	`%dl`	3rd argument
`%rsi`	`%esi`	`%si`	`%sil`	2nd argument
`%rdi`	`%edi`	`%di`	`%dil`	1st argument
`%rbp`	`%ebp`	`%bp`	`%bpl`	Callee-saved (frame pointer)
`%rsp`	`%esp`	`%sp`	`%spl`	Stack pointer
`%r8`–`%r9`	`%r8d`–`%r9d`	`%r8w`–`%r9w`	`%r8b`–`%r9b`	5th and 6th arguments
`%r10`–`%r11`	`%r10d`–`%r11d`	`%r10w`–`%r11w`	`%r10b`–`%r11b`	Caller-saved scratch
`%r12`–`%r15`	`%r12d`–`%r15d`	`%r12w`–`%r15w`	`%r12b`–`%r15b`	Callee-saved

Important 6.8. 32-bit Zero-Extension: Writing to a 32-bit register (e.g., `movl`) automatically zeroes the upper 32 bits of the 64-bit register. Writing to 8-bit or 16-bit registers preserves the upper bits.

Definition 6.9. Memory Addressing: The general addressing mode $D(R_b, R_i, S)$ accesses memory at:

$$\text{Mem}[\text{Reg}[R_b] + S \cdot \text{Reg}[R_i] + D]$$

where D is a constant displacement, R_b is the base register, R_i is the index register (cannot be `%rsp`), and $S \in \{1, 2, 4, 8\}$ is the scale factor.

Definition 6.10. Data Movement (`mov`): Transfers data between immediates, registers, and memory (`movb`, `movw`, `movl`, `movq`). When moving a smaller value to a larger register:

- **`movz` (Zero-extension):** Fills upper bits with 0s (unsigned, e.g., `movzbl`).
- **`movs` (Sign-extension):** Replicates the sign bit into upper bits (signed, e.g., `movslq`, or `cltq` for `%eax` $\rightarrow$ `%rax`).

Important 6.11. No Memory-to-Memory Transfers: A single `mov` instruction cannot have both source and destination in memory (`movq (%rax), (%rdx)` is illegal); use an intermediate register instead.

6.3 Arithmetic and Logical Operations

Definition 6.12. Load Effective Address (`leaq S, D`): Uses memory addressing syntax to compute the address expression of `S` and stores the *value itself* into register `D` ($D \leftarrow \&S$) **without accessing memory** or modifying condition codes. Used for pointer generation and fast linear arithmetic ($x + k \cdot y + c$).

Code Example 6.13

```
1 movq (%rdi, %rcx, 8), %rax  // Memory read: rax = rdi[rcx] (reads 8
     B from memory)
2 leaq (%rdi, %rcx, 8), %rax  // Address calc: rax = rdi + 8*rcx (no
     memory access)
3 leaq (%rdi, %rdi, 2), %rax  // Fast arithmetic: rax = 3 * rdi
```

Definition 6.14. Standard Arithmetic & Logical Instructions:

- **Unary:** `incq D` ($D + 1$), `decq D` ($D - 1$), `negq D` ($-D$), `notq D` ($\sim D$).
- **Binary:** `addq S, D` ($D \leftarrow D + S$), `subq S, D` ($D \leftarrow D - S$), `imulq S, D` ($D \leftarrow D \times S$).
- **Logical:** `xorq`, `andq`, `orq`. (`xorl %eax, %eax` is the standard idiom to set `%rax` $= 0$).
- **Shifts:** `salq`/`shlq` (left shift), `sarq` (arithmetic right shift, sign-extends), `shrq` (logical right shift, zero-fills). Variable shift counts must be in `%cl`.

6.4 Condition Codes, `setX`, and `jX`

Definition 6.15. Condition Codes: Single-bit flags set implicitly by arithmetic instructions (except `leaq`) or explicitly by comparison instructions:

- **CF (Carry Flag):** Unsigned overflow (carry out of MSB).
- **ZF (Zero Flag):** Result is zero ($== 0$).
- **SF (Sign Flag):** Result is negative (MSB $== 1$).
- **OF (Overflow Flag):** Signed two's-complement overflow.

Definition 6.16. Explicit Comparisons (`cmp` and `test`): Update condition codes without altering operands:

- **`cmpq S2, S1`:** Sets flags based on $S_1 - S_2$ (note AT&T operand order: compares S_1 with S_2).
- **`testq S2, S1`:** Sets flags based on S_1 & S_2 (e.g., `testq %rax, %rax` checks if `%rax` is 0 or negative).

Definition 6.17. Reading Condition Codes (`setX`) and Branching (`jX`):

- **`setX D`:** Sets a single low-order byte register (e.g., `%al`) to `1` if condition `X` holds, or `0` otherwise (follow with `movzbl %al, %eax` to clear upper bits).
- **`jX Label`:** Conditionally jumps to `Label` if condition `X` holds (`jmp` for unconditional jump).

Suffix (`X`)	Condition	Flags Checked	Description
`e` / `z`	$==$ / Zero	ZF	Equal / Zero
`ne` / `nz`	$\neq$ / Non-zero	$\sim$ ZF	Not equal / Not zero
`s` / `ns`	< 0 / ≥ 0	SF / $\sim$ SF	Negative / Non-negative
`g`	Signed $>$	$\sim$ (SF $\oplus$ OF) & $\sim$ ZF	Greater (signed)
`ge`	Signed $\geq$	$\sim$ (SF $\oplus$ OF)	Greater or equal (signed)
`l`	Signed $<$	SF $\oplus$ OF	Less (signed)
`le`	Signed $\leq$	(SF $\oplus$ OF) | ZF	Less or equal (signed)
`a`	Unsigned $>$	$\sim$ CF & $\sim$ ZF	Above (unsigned)
`ae`	Unsigned $\geq$	$\sim$ CF	Above or equal (unsigned)
`b`	Unsigned $<$	CF	Below (unsigned)
`be`	Unsigned $\leq$	CF | ZF	Below or equal (unsigned)

7 Compiling C Control Flow and Procedures

7.1 Conditionals: Branching vs. Conditional Moves

Definition 7.1. Compiling Control Flow: C control flow constructs (`if-else`, loops, `switch`) are translated into assembly using **conditional jumps** (`jX`), **unconditional jumps** (`jmp`), and **conditional moves** (`cmovX`).

Definition 7.2. Conditional Jumps vs. Conditional Moves:

- **Conditional Jumps (`jX`):** Transfer control by modifying `%rip` when the condition holds.
- **Conditional Moves (`cmovX S, D`):** Eagerly evaluate *both* branches and copy `S` to `D` only if condition `X` holds, avoiding control flow jumps and branch misprediction penalties.

Code Example 7.3

```
// C code: long absdiff(long x, long y) { return (x < y) ? (y - x) : (x - y); }

// 1. Branch-based:                    // 2. Conditional Move (cmov):
absdiff_branch:                        absdiff_cmov:
    cmpq  %rsi, %rdi                       movq  %rsi, %rdx
    jl    .L_then                          subq  %rdi, %rdx    // rdx = y - x
    movq  %rdi, %rax                       movq  %rdi, %rax
    subq  %rsi, %rax                       subq  %rsi, %rax    // rax = x - y
    ret                                    cmpq  %rsi, %rdi
.L_then:                                   cmovl %rdx, %rax    // if x < y: rax = rdx
    movq  %rsi, %rax                       ret
    subq  %rdi, %rax
    ret
```

Important 7.4. `cmov` cannot be used if either branch has side effects (e.g., `x++`), is expensive to compute, or may fault when evaluated eagerly (e.g., null dereference `p ? *p : 0`).

7.2 Loops and Switch Statements

Definition 7.5. Loop Translations:

- **`do-while` (Backward-Branching):** Executes the body first, then evaluates the condition at the bottom with a conditional jump backward (`jX .L_body`).

- **`while` (Jump-to-Middle):** Jumps unconditionally (`jmp .L_test`) to the test at the bottom; if true, branches back to the top of the body.
- **`while` (Guarded-Do):** Tests the condition initially to skip the loop if false, then runs as a backward-branching `do-while` loop.
- **`for (init; test; update)`:** Compiled into the same structure as a `while` loop with `init` before the loop and `update` at the end of the body.

Code Example 7.6

```
// Jump-to-Middle while loop:
    jmp   .L_test
.L_body:
    Initial guard
    imulq %rdi, %rax
    subq  $1, %rdi
.L_test:
    cmpq  $1, %rdi
    jg    .L_body
        Backward branch
```

```
// Guarded-Do while loop:
    cmpq  $1, %rdi
    jle   .L_done        //
.L_body:
    imulq %rdi, %rax
    subq  $1, %rdi
    cmpq  $1, %rdi
    jg    .L_body        //
.L_done:
```

Definition 7.7. Switch Statements:

- **Compact / Contiguous Cases → Jump Table ($O(1)$):** Stores code label addresses in an array in memory (`.rodata`). Normalizes the index ($n - \min$), checks bounds with a single unsigned comparison (`ja .L_default`), and performs an indirect jump: `jmp *.L_table(,%rdi,8)`.
- **Sparse / Non-Contiguous Cases → Decision Tree ($O(\log N)$):** When cases are far apart, a jump table wastes memory; the compiler instead emits a binary search tree of `cmp` and conditional branches.

7.3 Procedures and the Calling Convention

Definition 7.8. The Stack and Control Transfer: The x86-64 stack grows **downward** toward lower memory addresses, with `%rsp` pointing to the current top of the stack.

- **`pushq S`:** `%rsp` $\leftarrow$ `%rsp` $- 8$, then Mem[`%rsp`] $\leftarrow S$.
- **`popq D`:** $D \leftarrow$ Mem[`%rsp`], then `%rsp` $\leftarrow$ `%rsp` $+ 8$.
- **`call Target`:** Pushes the return address onto the stack and jumps to `Target`.
- **`ret`:** Pops the return address off the stack into `%rip` to restore instruction flow.

Theorem 7.9. x86-64 Linux Calling Convention (System V ABI):

- **Arguments 1–6:** Passed in `%rdi`, `%rsi`, `%rdx`, `%rcx`, `%r8`, `%r9` (floats in `%xmm0`–`%xmm7`).
- **Arguments 7+:** Pushed onto the stack in reverse order (7th arg at `(%rsp)` before `call`).
- **Return Value:** Passed back in `%rax`.
- **Caller-Saved Registers (`%rax`, `%rdi`, `%rsi`, `%rdx`, `%rcx`, `%r8`–`%r11`):** Temporary registers that can be freely clobbered by the callee; the caller must save them before `call` if needed afterward.
- **Callee-Saved Registers (`%rbx`, `%rbp`, `%r12`–`%r15`, `%rsp`):** Must be preserved across calls; if the callee uses them, it must push them on entry and pop them before `ret`.

Code Example 7.10

```
compute:
    pushq %rbx              // Save callee-saved %rbx
    movq  %rdi, %rbx        // Keep x safe across function call
    call  helper            // helper(x) -> result in %rax
    addq  %rbx, %rax        // return helper(x) + x
    popq  %rbx              // Restore %rbx
    ret
```

7.4 Stack Frame Optimizations and Variadic Functions

Remark 7.11. Frame Pointer Omission (`-fomit-frame-pointer`): Instead of maintaining `%rbp` as a dedicated base frame pointer, modern x86-64 compilers compute stack frame sizes at compile time and access all local stack variables directly relative to `%rsp`, freeing `%rbp` for general-purpose use.

Definition 7.12. The Red Zone: A 128-byte region immediately below `%rsp` (`-128(%rsp)` to `-1(%rsp)`) that the OS guarantees will not be overwritten by signals or interrupts. Leaf functions and small procedures can store temporary local data here without incrementing/decrementing `%rsp`.

Definition 7.13. Tail Call Optimization (TCO): When a function's final action is returning the result of another function call (`return f(x);`), the compiler reuses the current stack frame and replaces `call f; ret` with a direct unconditional jump (`jmp f`), avoiding extra stack frame allocation and call/return overhead.

Definition 7.14. Variadic Functions (`<stdarg.h>`): Functions taking a variable number of arguments (e.g., `printf`) use compiler intrinsics to iterate through arguments passed in registers and on the stack:

- **`va_list`:** Cursor structure tracking the current argument position.
- **`va_start(ap, last_named)`:** Initializes `ap` to point to the first variable argument.
- **`va_arg(ap, type)`:** Fetches the next argument of type `type` and advances `ap`.
- **`va_end(ap)`:** Cleans up the `va_list` before the function returns.

8 Compiling C Data Structures

8.1 Basic Data Types and Machine Representation

Definition 8.1. Primitive Data Types in x86-64 Linux (System V ABI): Primitive types have fixed sizes and alignment requirements:

Type	Size	Alignment	Suffix	Description
`char`	1 B	1 B	`b`	Byte
`short`	2 B	2 B	`w`	Word (16 bits)
`int`	4 B	4 B	`l`	Doubleword (32 bits)
`long`	8 B	8 B	`q`	Quadword (64 bits)
`long long`	8 B	8 B	`q`	Quadword (64 bits)
`T *` (pointer)	8 B	8 B	`q`	Quadword address
`float`	4 B	4 B	`s` / `ss`	Single precision (SSE)
`double`	8 B	8 B	`l` / `sd`	Double precision (SSE)
`long double`	16 B	16 B	`t`	Extended precision

Important 8.2. Assembly code does not retain high-level C types. All data is viewed merely as untyped byte sequences in registers or flat virtual memory.

8.2 One-Dimensional Arrays

Definition 8.3. An array declaration `T A[N]` allocates a **contiguous** block of $N \times L$ bytes, where $L = \text{sizeof}(T)$. The identifier `A` acts as a pointer pointing to the first element (`&A[0]`).

Important 8.4. C does not perform array bounds checking at compile-time or runtime. Accessing out-of-bounds indices reads or corrupts adjacent memory or triggers segmentation faults.

Definition 8.5. Indexing `A[i]` is semantically identical to pointer dereferencing `*(A + i)`. Given starting address x_A, the memory address of element i is $\&A[i] = x_A + L \cdot i$

Remark 8.6. Assembly Translation: Memory operands in x86-64 map directly to array indexing using the scaled index addressing mode $D(R_b, R_i, S)$:

$$\text{Mem}[\text{Reg}[R_b] + S \cdot \text{Reg}[R_i] + D]$$

where $S \in \{1, 2, 4, 8\}$ corresponds to element size L. The address of an element can be computed without reading memory using `leaq`.

8.3 Nested Arrays

Definition 8.7. A multidimensional array declaration `T A[R][C]` allocates a single contiguous memory block of $R \times C \times L$ bytes, where R is rows, C is columns, and $L = \text{sizeof}(T)$.

Remark 8.8. C stores nested arrays row-by-row sequentially. All elements of row 0 are stored first, followed contiguously by row 1, row 2, and so forth.

Theorem 8.9. Address Computation: The starting address of row i (`A[i]`) is $\&A[i][0] = x_A + i \cdot (C \cdot L)$ and the memory address of element `A[i][j]` is $\&A[i][j] = x_A + L \cdot (i \cdot C + j) = x_A + (i \cdot C \cdot L) + (j \cdot L)$

Remark 8.10. Compiler Optimizations: When the column count C is a constant or a power of two, the compiler avoids costly multiplication instructions (`imulq`) by replacing $i \cdot C$ with bit shifts (`salq`) and scaled addition (`leaq`).

8.4 Multi-Level Arrays

Definition 8.11. A **multi-level array** declaration `T *B[R]` allocates an array of R contiguous pointers (each 8 bytes on x86-64), where each entry `B[i]` holds a pointer to a separate, independently allocated 1D array of elements of type `T`.

Important 8.12. Rows are not required to be contiguous to each other and can have variable lengths (ragged/jagged arrays).

8.5 Structures and Data Alignment

Definition 8.13. Structures (`struct`):

- A `struct` groups heterogeneous data fields into a single contiguous memory block.
- Field order is **strictly preserved** as declared in the C source code.

- The compiler computes fixed byte offsets from the base address for each field at compile time.
- Accessing a struct field compiles to base-plus-displacement addressing: `movl D(%rdi), %eax`.

Remark 8.14. Modern CPU memory architectures and cache lines (typically 64 bytes) fetch data from aligned addresses (multiples of 2, 4, 8, or 64 bytes). If data crosses a cache line or virtual page boundary, accessing it requires multiple memory bus cycles, severely degrading memory bandwidth and CPU performance.

Theorem 8.15. K-byte Alignment Rule: Any primitive object of size K bytes must reside at a memory address that is a multiple of K:

$$\text{Address} \equiv 0 \pmod{K}$$

Theorem 8.16. Structure Alignment and Padding Rules:

- **Overall Struct Alignment (K_{struct}):** The alignment of the entire struct is dictated by the maximum alignment required by any of its members:

  $$K_{\text{struct}} = \max_i(K_{\text{field}_i})$$

  The struct's starting address must be a multiple of K_{struct}.
- **Internal Padding:** Unused gap bytes inserted between adjacent fields so that each individual field starts at an offset satisfying its own alignment constraint.
- **External (Trailing) Padding:** Unused gap bytes appended to the end of the struct so that the total struct size is an exact multiple of K_{struct}.

Important 8.17. Why External Padding is Mandatory: External padding ensures that when structs are placed into an array (`struct S arr[N]`), every element `arr[i]` starts at an offset that satisfies K_{struct}. Without trailing padding, the second element `arr[1]` would violate alignment.

Remark 8.18. Because field order is preserved by the compiler, poor ordering introduces wasted internal padding. Ordering fields in **descending order of size** eliminates or minimizes internal padding.

8.6 Unions

Definition 8.19. Unions (`union`):

- A `union` allocates all of its member fields at the exact same starting memory address (offset 0), effectively overlaying them.
- Only one member field can be validly stored at any given point in time.
- **Size and Alignment:** The total size of a union equals the size of its largest member, rounded up to a multiple of its largest alignment constraint ($K_{\text{union}} = \max_i(K_{\text{field}_i})$).